CLOUD SECURITY – CAPSTONE PROJECT REPORT

Deploy a Secure Three-Tier Application on AWS Using Terraform

Student Name: Suman Yadav

PRN: 2125UCSM2003

Course / Class: Cyber Security

Submitted To: Mr.Prashant Kumar

Date: 20-10-2026

1. Abstract

This project demonstrates a secure three-tier web application architecture on Amazon Web Services (AWS), provisioned using Terraform Infrastructure as Code (IaC). The design separates the public entry point, application compute, and database into different network tiers. Security groups restrict communication between tiers, the RDS database is not publicly accessible, and monitoring is enabled through VPC Flow Logs and AWS CloudTrail.

2. Objectives

- Create a VPC with public and private subnets across two Availability Zones.
- Place an Application Load Balancer in public subnets.
- Run application EC2 instances in private application subnets.
- Deploy an encrypted MySQL RDS database in isolated private database subnets.
- Apply least-privilege security-group rules and routing.
- Collect network-flow and API audit logs.
- Use Terraform to make infrastructure repeatable and reviewable.

3. Tools and Technologies

- Amazon VPC, EC2, Application Load Balancer, Auto Scaling, RDS, S3, CloudWatch Logs, CloudTrail, IAM.
- Terraform 1.5 or later and AWS provider 5.x.
- Amazon Linux 2023 and Apache HTTP Server for the sample web tier.
- AWS CLI for credentials and deployment.

4. Architecture

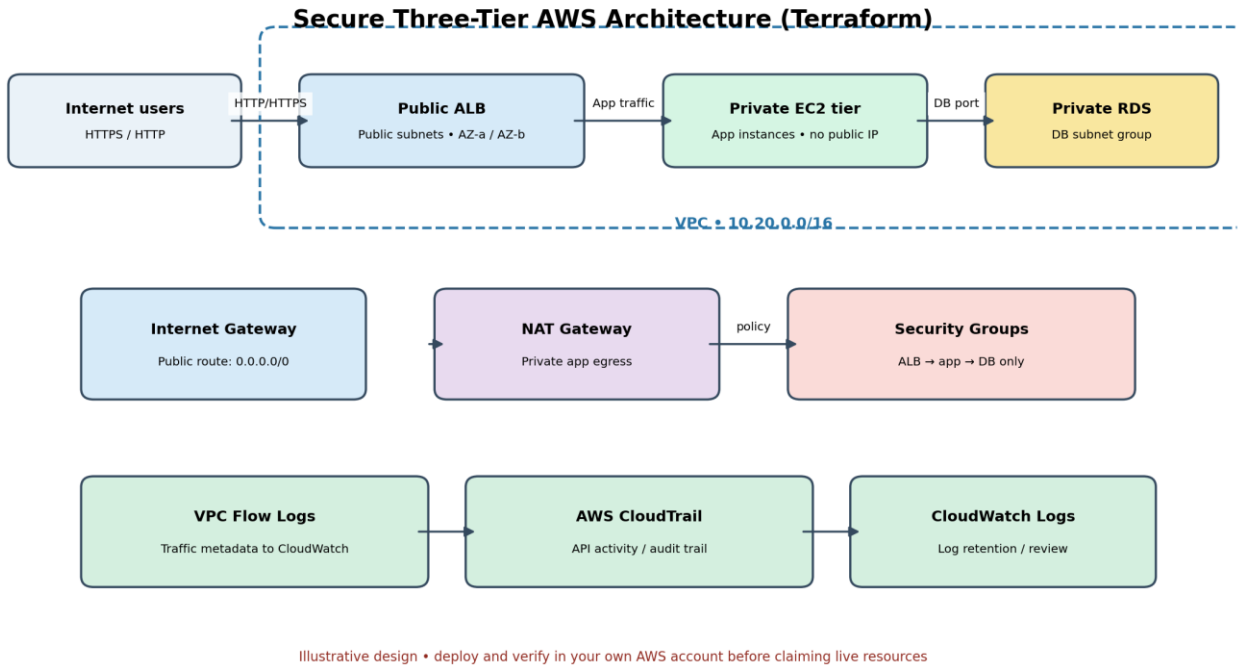


Figure 1. Proposed architecture. Public users connect to the ALB. The ALB forwards web requests to EC2 instances in private subnets. The application security group is the only permitted source for MySQL traffic to RDS. Flow Logs and CloudTrail provide monitoring and audit evidence.

5. Network and Security Design

- VPC CIDR: 10.20.0.0/16.
- Public subnets: 10.20.1.0/24 and 10.20.2.0/24; contain the ALB and NAT Gateway.
- Private app subnets: 10.20.11.0/24 and 10.20.12.0/24; contain EC2 instances without public IP addresses.
- Private database subnets: 10.20.21.0/24 and 10.20.22.0/24; have no default internet route.
- ALB security group accepts TCP/80 for the demonstration. The application security group accepts TCP/80 only from the ALB security group.
- The database security group accepts TCP/3306 only from the application security group.
- RDS uses encrypted storage and is configured with publicly_accessible = false.
- The example uses one NAT Gateway to reduce cost; production deployments may use one per AZ for higher availability.

6. Implementation Procedure

Step 1 – Prepare tools

Install Terraform and AWS CLI. Configure an AWS identity with the permissions required to create and manage the listed resources. Enable a billing alert before deployment.

Step 2 – Review variables

Open terraform/variables.tf and terraform/terraform.tfvars.example. Set the region and instance sizes as needed. Supply the database password through the TF_VAR_db_password environment variable; do not commit secrets.

Step 3 – Initialize and validate

From the terraform folder, run terraform init, terraform fmt -recursive, terraform validate, and terraform plan. Review the plan carefully.

Step 4 – Deploy

Run terraform apply and approve only after reviewing all planned resources. Wait for the ALB target group health checks to pass.

Step 5 – Verify

Open the ALB DNS output in a browser. Verify EC2 instances are private, RDS says Publicly accessible: No, and security-group sources match the design.

Step 6 – Inspect monitoring

Open CloudWatch Logs for the VPC Flow Logs group and CloudTrail Event history. Save genuine screenshots showing resource state and configuration.

Step 7 – Clean up

After collecting evidence, run terraform destroy and confirm the plan. Check the AWS console afterward to make sure resources were removed.

7. Terraform Source Code

The accompanying ZIP includes versions.tf, variables.tf, main.tf, outputs.tf and an example tfvars file. The Terraform source provisions the VPC, subnet and route design, security groups, ALB, Auto Scaling Group, RDS, Flow Logs, and CloudTrail.

8. Testing and Verification Plan

Test	Expected result	Evidence to capture
ALB webpage	Demo page loads through ALB DNS	Browser + ALB target health
EC2 placement	Instances have no public IP and use private app subnets	EC2 instance details
RDS public exposure	Publicly accessible = No	RDS connectivity tab
Database ingress	TCP 3306 source is app SG only	DB security group inbound rules
Application ingress	TCP 80 source is ALB SG only	App security group inbound rules
VPC Flow Logs	Flow log active and log group receives events	VPC Flow Logs + CloudWatch
CloudTrail	Trail enabled and management events available	CloudTrail trail details / Event history
Cleanup	No project resources remain after destroy	Terraform output + console check

9. Screenshot Evidence

The folder screenshots_illustrative contains six console-style mockups showing the expected configuration. These are clearly marked as illustrative and are not live AWS screenshots. Replace or supplement them with genuine screenshots captured from your own account after deployment.

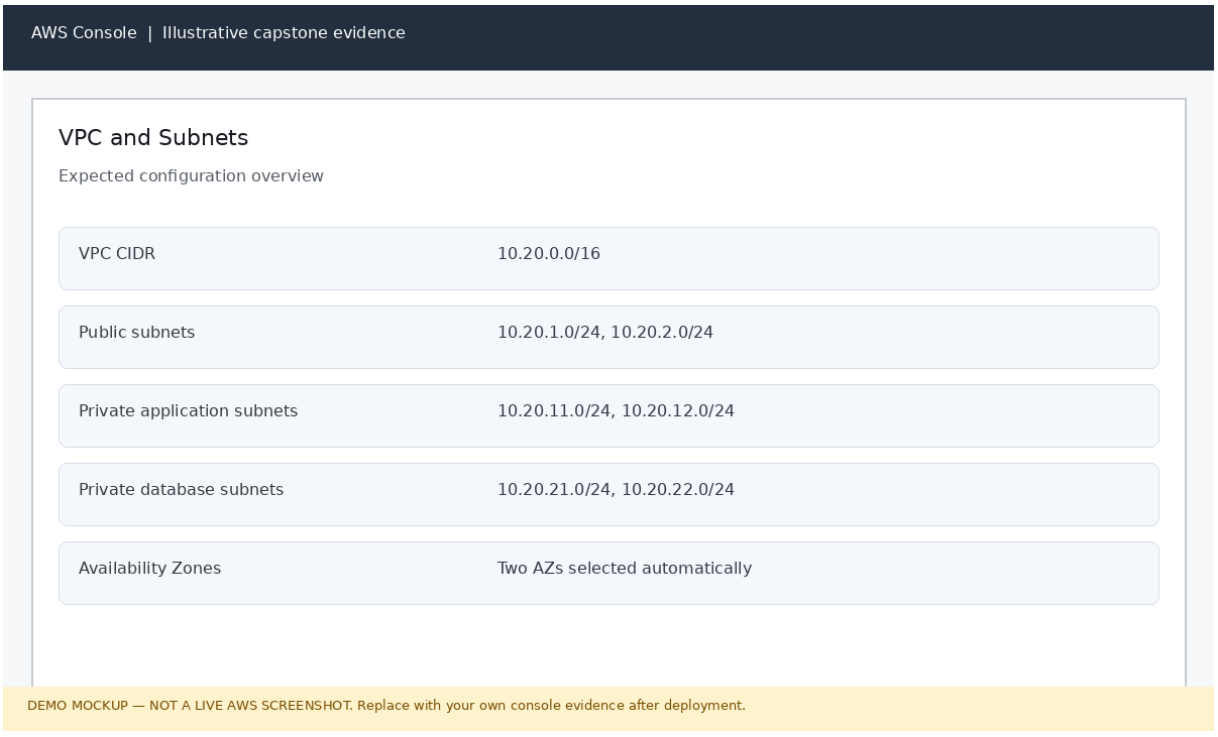


Figure 2. VPC and subnet layout – illustrative mockup

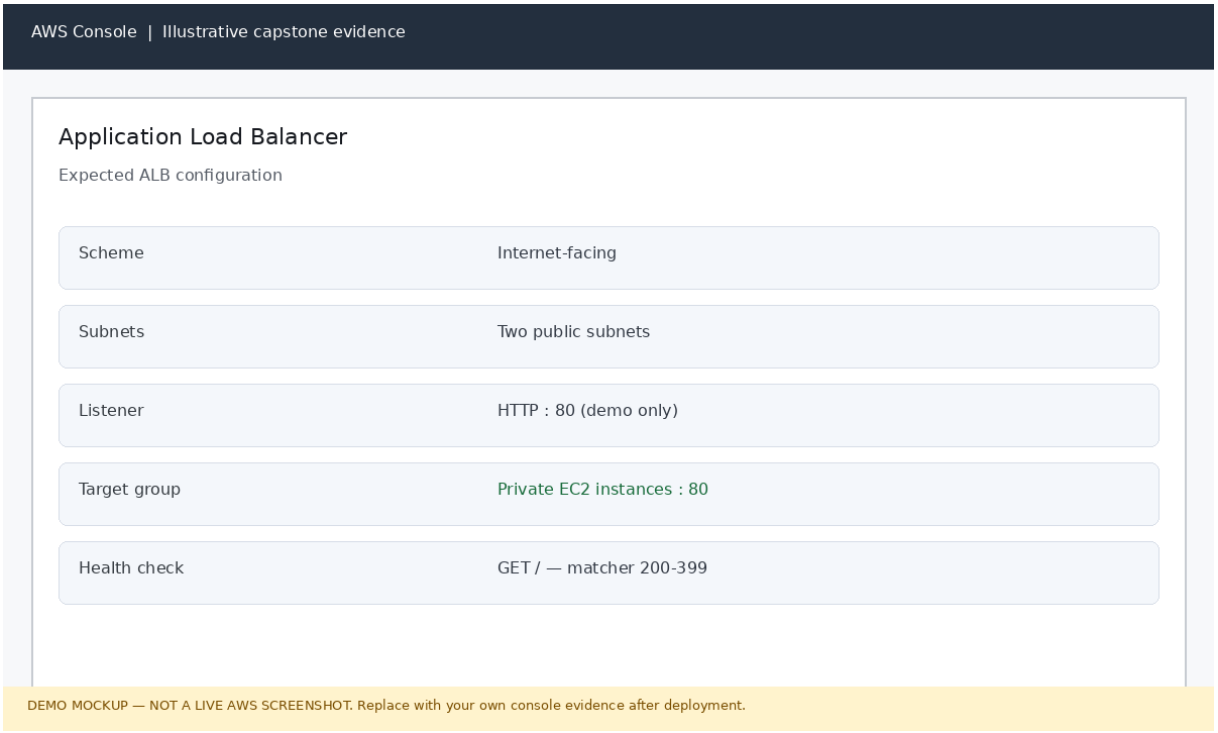


Figure 3. ALB configuration – illustrative mockup

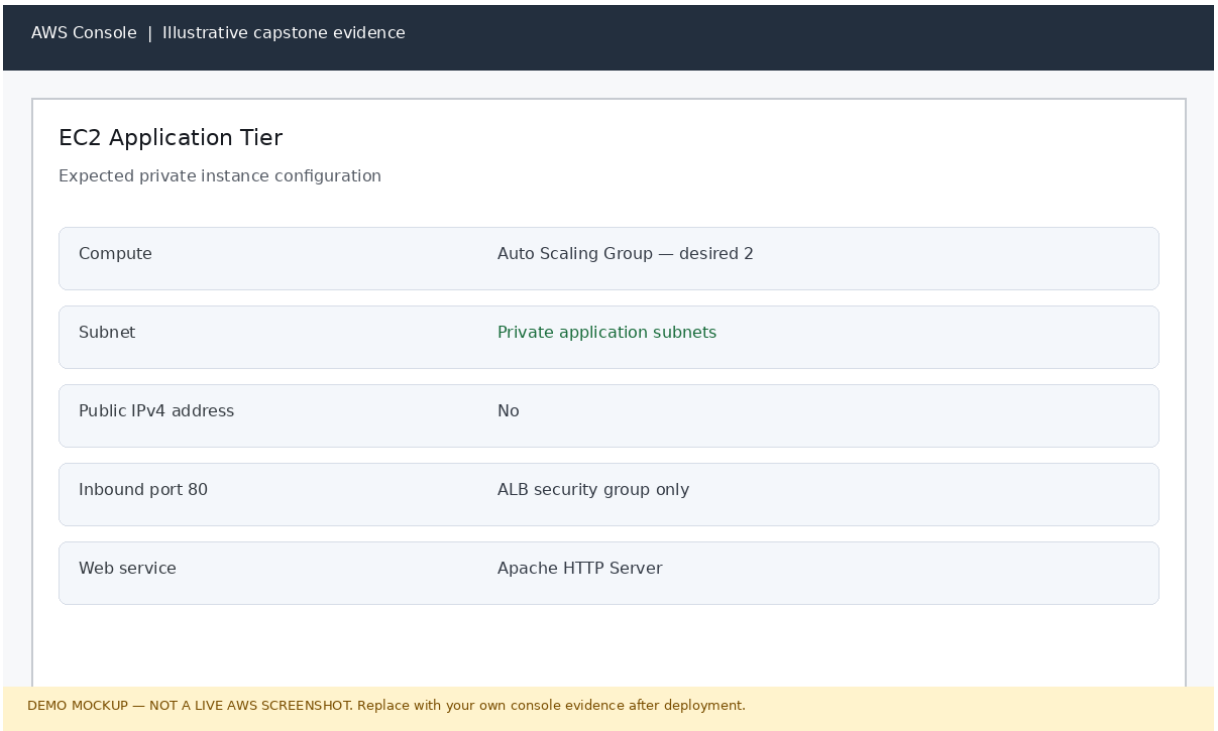


Figure 4. Private EC2 tier – illustrative mockup

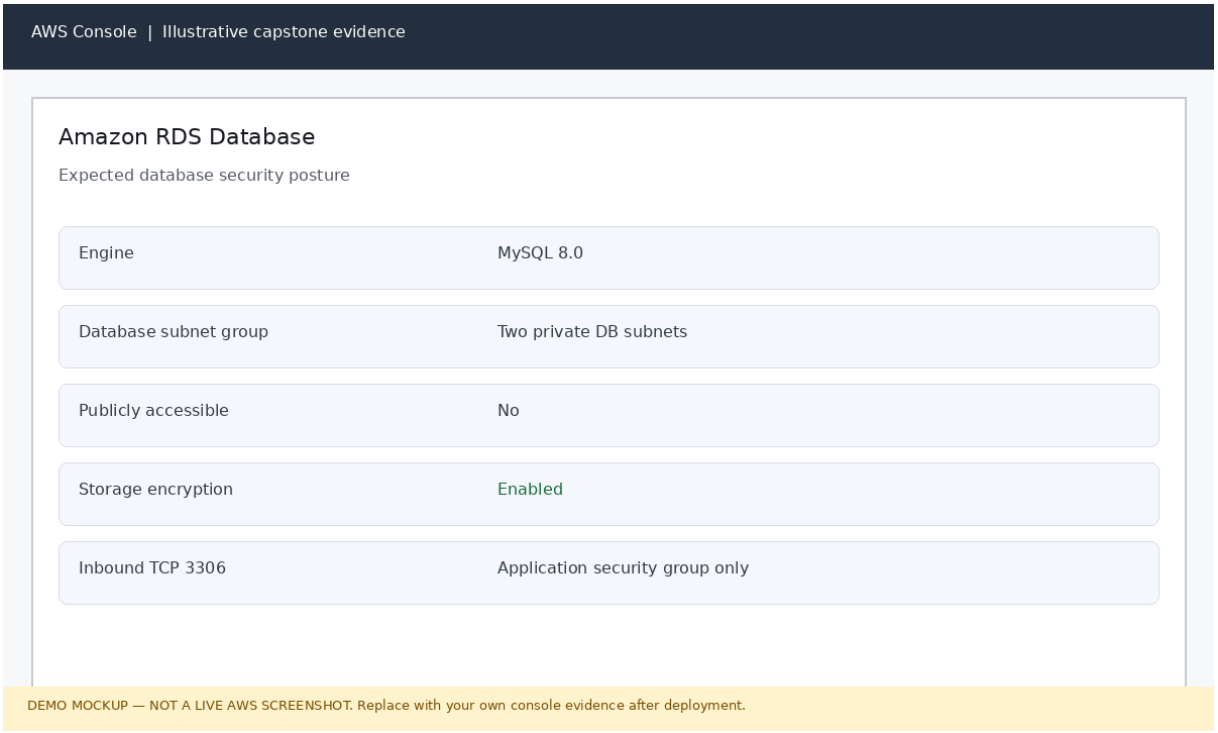


Figure 5. RDS private configuration – illustrative mockup

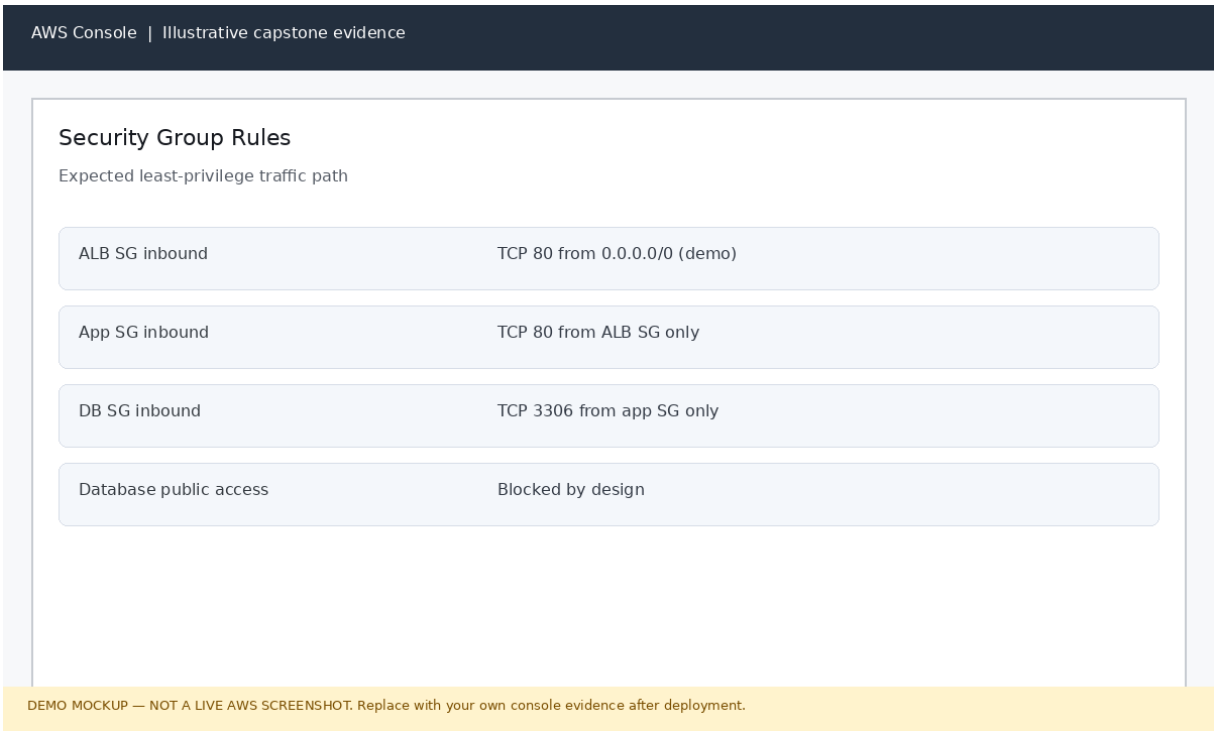


Figure 6. Security group traffic rules – illustrative mockup

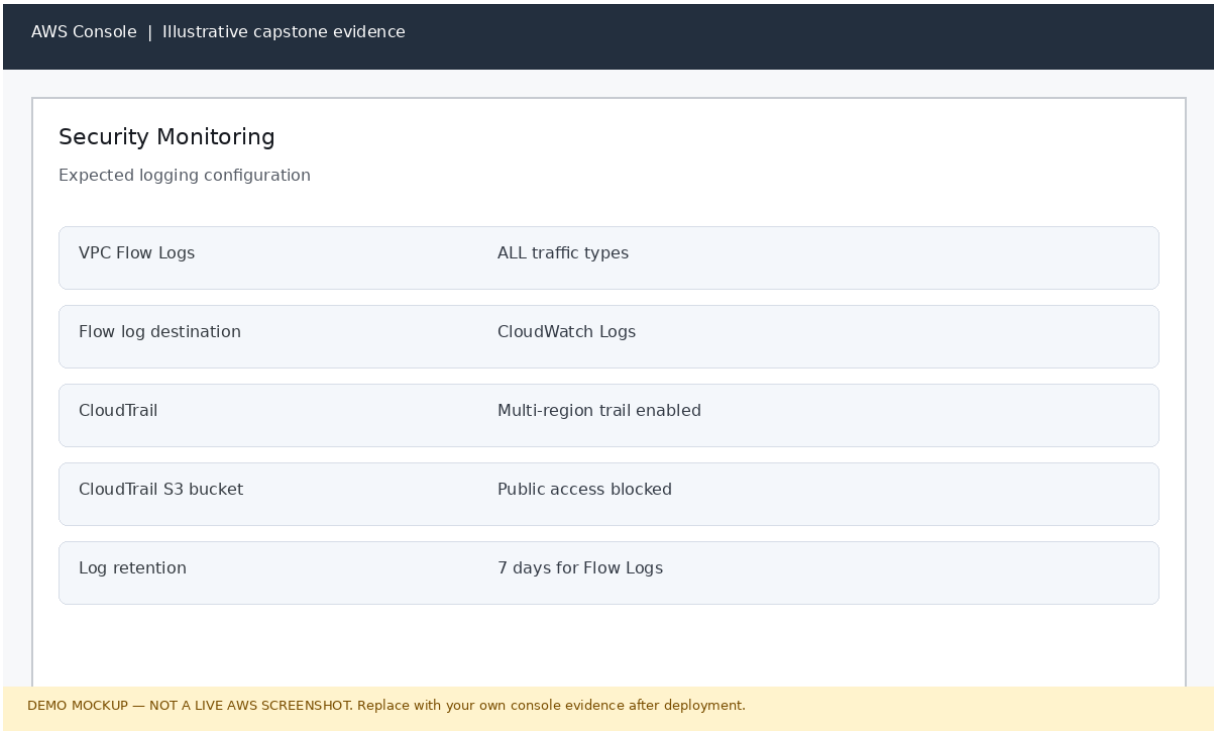


Figure 7. Monitoring configuration – illustrative mockup

10. Results

The Terraform configuration is designed to create the requested three-tier architecture. Actual deployment success, target health, log delivery, and resource IDs must be confirmed by running the code in an AWS account. No live deployment is claimed by this report template.

11. Limitations

- The listener is HTTP-only for simple demonstration; it should be replaced with HTTPS and an ACM certificate for production.
- The single NAT Gateway is a cost-saving demonstration choice and a potential availability bottleneck.
- The sample web page does not implement application-to-database queries; the database is provisioned to demonstrate network isolation.
- A successful Terraform plan does not guarantee apply success; region, quotas, permissions, and available engine versions can affect deployment.
- Resources incur charges while running; cleanup is essential.

12. Future Scope

- Add HTTPS using AWS Certificate Manager and redirect HTTP to HTTPS.
- Use AWS Secrets Manager for database credentials and rotate secrets.
- Add AWS WAF, GuardDuty, Security Hub, and CloudWatch alarms.
- Use Systems Manager Session Manager instead of SSH access.
- Add Multi-AZ RDS, separate NAT Gateways per AZ, and automated backup/restore testing.
- Add CI/CD, Terraform remote state with locking, and policy-as-code checks.

13. Conclusion

This capstone shows how Infrastructure as Code can build a segmented AWS environment with a public load balancer, private application compute, an isolated database, and centralized security logging. The key security outcome is that the database has no public access and accepts MySQL connections only from the application tier. The architecture should be deployed, tested, and cleaned up in a controlled AWS account before final submission.

14. References

- AWS Documentation – Amazon VPC User Guide: https://docs.aws.amazon.com/vpc/
- AWS Documentation – Elastic Load Balancing: https://docs.aws.amazon.com/elasticloadbalancing/
- AWS Documentation – Amazon RDS: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/
- AWS Documentation – AWS CloudTrail: https://docs.aws.amazon.com/awscloudtrail/
- AWS Documentation – VPC Flow Logs: https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html
- HashiCorp Terraform AWS Provider Documentation: https://registry.terraform.io/providers/hashicorp/aws/latest/docs